

**Student Name:**

Minh Quan Tran

Susheela Narayan Hegde

**Team Name:**

Bin There, Done That

**Project Name:**

Smart parcel sorting system



# SMART PARCEL SORTING SYSTEM

# LOGBOOK WEEK 3

## 1. Goal Past Week

- Write the ESP32 code to subscribe receive data from cloud and print it out.
- Expand the Raspberry Pi Python code to both publish to AWS IoT topics.
- Integrate both systems to establish a working prototype:
  - Raspberry Pi reads and sends QR code data to AWS IoT.
  - ESP32 receives the data from AWS IoT and prints it to the serial monitor.
- Begin planning the hardware requirements by drafting a parts list.

## 2. Achievement

- Successfully expanding Raspberry Pi Python code to both read the QR data and publish to AWS IoT topics



Figure 1: Tested QR code

```
File Edit Tabs Help
ESD_Group1@ESDGroup1: ~/Capstone/Integration_QR_AWSIoT $ python3 main.py
[0:03:26.889409096] [2056] INFO Camera camera_manager.cpp:326 libcamera v0.5.0+59-d83ff0a4
[0:03:26.917322109] [2060] WARN RPiSdn sdn.cpp:40 Using legacy SDN tuning - please consider moving SDN inside rpi.denoise
[0:03:26.919573093] [2060] INFO RPI vc4.cpp:447 Registered camera /base/soc/i2c0mux/i2c@1/imx708@1a to Unicam device /dev/media1
and ISP device /dev/media0
[0:03:26.919716871] [2060] INFO RPI pipeline_base.cpp:1121 Using configuration file '/usr/share/libcamera/pipeline/rpi/vc4/rpi_ap
ps.yaml'
[0:03:26.932541497] [2056] INFO Camera camera.cpp:1205 configuring streams: (0) 640x480-RGB888 (1) 1536x864-SBGGR10_CSI2P
[0:03:26.933163905] [2060] INFO RPI vc4.cpp:622 Sensor: /base/soc/i2c0mux/i2c@1/imx708@1a - Selected sensor format: 1536x864-SBGG
R10_1X10 - Selected unicam format: 1536x864-pBAA
Starting QR code scan (up to 10 attempts)...
Attempt 1...
Attempt 2...
Attempt 3...
Attempt 4...
QR Code Detected: blue
Connecting to AWS IoT at 'aqrm5lymlussz-ats.iot.ca-central-1.amazonaws.com'...
Connected to AWS IoT
Published to topic 'esp32/sub':
{
  "message": "blue"
}
Disconnected from AWS IoT
ESD_Group1@ESDGroup1: ~/Capstone/Integration_QR_AWSIoT $
```

Figure 2: Output python script

```
load:0x40078000,len:16516
load:0x40080400,len:4
load:0x40080404,len:3476
entry 0x400805b4
Connecting to WiFi.....connected
Connecting to AWS IoT...connected
incoming: esp32/sub
blue
```

*Figure 3: ESP32 received the QR data*

- List out all the hardware part that will be required for the project and submitted to econestoga.

### **3. Goal for Week 4**

- Received hardware from tool room and start testing each component making sure it will work properly
- Draft state machine for all STM32, ESP32, Raspberry Pi 4.